DevOps 课程学习完全指南

从零基础到独立完成全部练习项目

Git 版本控制 · 软件测试与 FastAPI · GitHub Actions · Docker

配套材料：笔记 / 课件 / 练习指南（含中文版）/ cars-crud 练习项目

适用对象：未接触过 FastAPI、Docker、Git 版本控制，部分编程基础语法需要回顾的学习者

2026 年 10 月

目录

页码为打开文档前的估计值；在 Word 中右键目录选择“更新域”即可自动刷新为准确页码。

第一章 导学：这门课在学什么、怎么学 6
1.1 DevOps 是什么 6
衡量 DevOps 水平的四大指标 6
DevOps 全流程工具链（与本课程的对应关系） 6
1.2 课程材料地图 6
1.3 给零基础学习者的学习路线 7
第二章 预备知识补课（地基） 8
2.1 命令行终端基础 8
必会的目录与文件命令 8
2.2 Python 语法速览 8
运行环境与虚拟环境 8
看懂 cars-crud 用到的语法点 9
pytest 与 fixture（预习） 9
2.3 深入：装饰器（@ 到底是什么） 10
准备工作一：名字只是标签 10
准备工作二：函数里面可以定义函数（闭包） 10
手动装饰：一步一步拆解 10
第二步：@ 语法糖只是自动帮你写那一行 11
能处理任意参数的标准写法：再拆一遍 12
带参数的装饰器（看懂即可） 13
先补一课：“.” 号连接的是什么 13
functools.wraps 内部到底包装了什么 13
“__name__”这种两头双下划线的名字是什么意思 14
课程中的三个真实装饰器 14
pytest 的装饰器内部做了什么（@pytest.fixture / @pytest.mark.parametrize） 15
2.4 深入：类与对象 16
最小例子：定义一个类 16
继承：在已有类的基础上扩展 16
Pydantic：用“声明”代替“手工” 16
创建对象时，Pydantic 内部按什么顺序干活 17
model_dump()：对象转字典 18
回到 FastAPI：一个声明在三个地方生效 18
类变量与实例变量（了解） 18

2.5 HTTP 基础：请求方法、状态码与常见名词 18
HTTP 请求方法（动词） 19
常见状态码 19
其他名词 19
2.6 Python Web 开发入门 19
Web 应用的本质：一台永远等着回话的机器 19
最小的 FastAPI 应用：只有 5 行 20
路由：把 URL 映射到函数 20
响应：函数 return 的东西就是浏览器收到的东西 20
依赖注入：FastAPI 最优雅的设计 21
一个请求在 cars-crud 里的完整旅程 21
前端页面从哪来：三种主流方案 21
2.7 YAML 基础（GitHub Actions 用） 22
2.8 环境安装检查清单 22
第三章 Git 与 GitHub 版本控制 22
3.1 为什么需要版本控制 22
3.2 核心概念 23
3.3 常用命令速查 23
日常命令 23
容易混淆的三组命令 24
3.4 配套练习逐步讲解（git-guide-and-practice 的 15 个练习） 24
练习 1-2：创建空仓库并把本地文件夹连接到 GitHub 24
练习 3：检查、暂存、提交并推送一次编辑 25
练习 4：创建分支并合并 25
练习 5：触发并解决合并冲突 25
练习 6：触发并解决 rebase 冲突（选学，课堂建议学到第 6 页即可） 26
练习 7：创建并合并一个 Pull Request 27
练习 8：fetch 与 pull 的区别 27
练习 9：克隆第二个副本，模拟队友协作 27
练习 10：pull 的另一种姿势——合并 28
练习 11：stash 搁置未完成的工作 28
练习 12：用 revert 撤销已发布的提交 28
练习 13：用 reset 重做未发布的提交 29
练习 14：取消暂存一个编辑，然后丢弃它 29
练习 15：收尾 29
3.5 冲突解决通用流程（背这个就够了） 29

3.6 常见错误速查 30
第四章 软件测试与 FastAPI 30
4.1 为什么要做自动化测试 30
三大测试类型（测试金字塔） 30
快乐路径与异常路径 30
测试驱动开发（TDD，了解即可） 31
4.2 pytest 入门 31
4.3 读懂 cars-crud 应用（逐文件讲解） 31
schemas.py：Pydantic 数据校验 31
database.py：SQLite 持久层 32
main.py：FastAPI 路由与依赖注入 32
4.4 四个测试层级实战（TESTING_WORKSHOP 练习） 33
动手前：启动应用 33
第 1 层：单元测试（不碰数据库和网络） 33
第 2 层：集成测试（应用代码 + 真实 SQLite） 33
第 3 层：API 测试（HTTP 契约） 34
第 4 层：E2E 测试（真实浏览器） 34
4.5 把测试流程搬到 GitHub（练习 2-1） 35
4.6 常见错误速查 35
第五章 GitHub Actions 持续集成 35
5.1 CI 是什么、GitHub Actions 怎么工作 35
5.2 最小工作流逐键讲解（练习 3-1） 36
5.3 实战：PR 上自动跑 Cars 测试（练习 3-2） 37
5.4 事件与分支过滤的细节 38
5.5 常见错误速查 38
第六章 Docker 容器化 38
6.1 为什么需要 Docker 38
镜像与容器 38
Docker 与虚拟机的区别 39
6.2 Dockerfile 逐行讲解（cars-crud） 39
6.3 构建、运行、端口映射与卷（练习 4-1 主体） 39
Docker CLI 速查表 40
6.4 国内加速：课程 Azure 代理 40
6.5 发布到 Docker Hub（练习 4-1 后半） 41
6.6 用 GitHub Actions 自动发布（练习 4-2） 41
第一步：在 GitHub 配置 secrets 41

第二步：改造工作流 41
验证闭环 42
6.7 Docker 常见错误速查 42
第七章 全链路串讲与复习 43
7.1 从写代码到上线：一张总图 43
7.2 全课程命令总表 43
Git 43
测试 44
Docker 44
7.3 自测题（附答案） 44
概念题 44
参考答案 44
7.4 考前与项目前检查清单 45

第一章 导学：这门课在学什么、怎么学

1.1 DevOps 是什么

DevOps 是 Development（开发）和 Operations（运维）的合成词。它不只是工具的堆砌，而是一种**团队协作理念和工作流程**：把“规划 → 编码 → 构建 → 测试 → 发布 → 部署 → 运维 → 监控”这条链路标准化、自动化，让多人团队能够高频、稳定地交付软件。可以把它类比为软件开发领域的福特流水线——每个环节都是标准化步骤，代码像产品一样在流水线上流动。

传统开发模式的痛点：瀑布式开发交付周期长达数月甚至一年；多人协作时靠手动合并文件，极易出错。DevOps 的应对思路是：先交付最小可行产品（MVP）快速上线，再通过一轮轮迭代叠加功能、修复问题。软件开发没有绝对的“完成”，生命周期是无限循环的持续迭代。

衡量 DevOps 水平的四大指标

指标	定义	优化方向
部署频率	单位时间内应用正式上线的次数	越高越好（如从每月 1 次提升到每日多次）
变更交付周期	从代码写完到功能正式上线、用户可用的总耗时	越短越好（自动化的目标：1 小时以内）
变更失败率	上线后出现故障的比例	越低越好（接近 0）
故障检测与修复平均耗时（MTTR）	从发现线上问题到修复补丁上线的总耗时	越短越好

DevOps 全流程工具链（与本课程的对应关系）

阶段	工具/实践	本课程对应章节
规划	Jira、Trello 等看板工具	课程项目管理（了解即可）
开发	Git 本地版本控制 + GitHub 远程协作	第三章
测试	pytest 自动化测试、Playwright E2E	第四章
持续集成（CI）	GitHub Actions 自动运行测试	第五章
打包与部署	Docker 镜像、Docker Hub 发布	第六章

1.2 课程材料地图

课程文件夹分为三部分，加上一个贯穿始终的练习项目 cars-crud（汽车信息增删改查应用）。建议对照使用：

材料	内容	用途
笔记/	6 份课堂笔记（docx）：DevOps 概念、Git×2、软件测试、GitHub Actions、Docker	每章的“课本”，先读笔记建立概念
课件/	4 份英文课件 PDF：What is DevOps、Version control、Software testing、Docker and containers	英文原文参考，quiz 考点来源
练习指南/	8 份练习指南 PDF（英文原版 + 中文版子文件夹）	每章的“实验手册”，照着动手做
cars-crud/	FastAPI 练习项目：应用代码 + 四层测试 + Dockerfile + GitHub Actions 工作流	贯穿第三到第六章的实操对象
git-playground/ 与 git-playground-peer/	Git 练习时创建的本地仓库（含远程连接）	第三章练习的劳动成果

1.3 给零基础学习者的学习路线

如果你完全没有接触过 Git、FastAPI、Docker，并且 Python 语法有些生疏，请按下面的顺序学习，不要跳步。每个阶段的目标都写成了“能做到什么”。

1. 第 0 步（半天）：先读本文档第二章，把命令行、Python 语法、Web 基础补起来。这一步不做任何练习，只看懂示例即可。
2. 第 1 步（第 1 天下午）：第三章 Git。先读课堂笔记 L2、L3，再对照本文档 3.4 节把 git-guide-and-practice 的 15 个练习全部做一遍。目标：能独立建仓库、提交、分支、合并、解决冲突、推送 GitHub、开 PR。
3. 第 2 步（第 2 天上午）：第四章测试。先读笔记 L4，再读懂 cars-crud 应用的三个源文件（本文档 4.3 节逐行讲解），然后按 4.4 节完成 TESTING_WORKSHOP 的四层测试练习，最后按 2-1 指南把测试通过 PR 合并进 main。
4. 第 3 步（第 2 天下午）：第五章 GitHub Actions。读笔记 261013L1，按本文档 5.3 节完成 tests.yml 工作流，看到 PR 上自动跑出绿色对勾。
5. 第 4 步（第 3 天）：第六章 Docker。读笔记 261013L2，按 6.4 节把 cars-crud 打包成镜像跑起来，再按 6.7 节完成 GitHub Actions 自动发布到 Docker Hub。
6. 第 5 步（复习）：做第七章的自测题，对照全课程命令总表查漏补缺。

> **学习建议**
>
> 每个练习的“预期失败”步骤都不是可有可无的：故意让测试失败、故意制造冲突，是为了让你亲眼看到错误长什么样，真正出问题时才不慌。
>
> 命令不需要死记硬背。真正要记的是**每条命令解决了什么问题**（本文档每个命令都附了“它在做什么”），用多了自然记住。
>
> 出错了先把错误信息读完整。本文档每章末尾都有“常见错误速查表”，90% 的问题都能在里面找到对应。

第二章 预备知识补课（地基）

本章为零基础读者补齐地基：命令行、Python 语法与装饰器、类与对象、HTTP 与 Python Web 开发、YAML。已经熟悉的内容可以直接跳过，但建议至少浏览一遍代码示例的风格，后面的章节会直接使用它们。

2.1 命令行终端基础

终端（Terminal）是通过输入文字命令来操作计算机的工具。课程中所有 Git、pytest、docker 命令都在终端里执行。Windows 上你可以使用 PowerShell 或 Git Bash（安装 Git 时自带），macOS/Linux 使用系统终端。

必会的目录与文件命令

命令	作用	示例
pwd	显示当前所在目录	pwd
ls (Windows: dir)	列出当前目录的文件	ls -l
cd 目录名	进入目录；cd .. 返回上一级	cd cars-crud
mkdir 名字	创建文件夹	mkdir git-playground
cat 文件名 (Windows: type)	显示文件内容	cat notes.txt
echo 文本 > 文件	创建/覆盖文件并写入内容	echo "hello" > a.txt
echo 文本 >> 文件	在文件末尾追加内容	echo "more" >> a.txt
rm 文件 (Windows: del)	删除文件	rm a.txt

其中 > 和 >> 是重定向符号：> 把输出写进文件（覆盖原有内容），>> 把输出追加到文件末尾。Git 练习里大量用 echo 和 > 快速创建练习文件，就是利用了这一点。

> **易错点**
>
> Windows PowerShell 里激活虚拟环境和运行脚本的方式与 macOS/Linux 不同，练习指南里一般会同时给出两套命令，注意选对。
>
> 路径中有空格时必须加引号，如 cd "My Projects"。
>
> 终端打开时的“当前目录”很重要。多数练习要求“在 cars-crud/ 目录下执行”，先 pwd 确认位置再执行命令。

2.2 Python 语法速览

运行环境与虚拟环境

课程要求 Python 3.11 或更高版本。检查版本：

```
python --version   # Windows 上也可以用 py --version
```

虚拟环境（venv）是项目专属的 Python 运行环境，不同项目的依赖互不影响。创建和激活：

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate

# Windows PowerShell
py -m venv .venv
..venvScriptspython.exe -m pip install -r requirements-dev.txt
# 激活后命令行前面会出现 (.venv) 字样，表示已在虚拟环境中
```

安装依赖用 pip（Python 的包管理器）：pip install -r requirements-dev.txt 表示按文件列表批量安装。项目里 requirements.txt 是运行应用所需的最小依赖，requirements-dev.txt 额外包含测试工具（pytest、playwright 等）。

看懂 cars-crud 用到的语法点

下面用课程项目里的真实代码解释你会遇到的语法。不必全部默写，能读懂即可。

语法点	例子（来自 cars-crud）	说明
导入模块	from fastapi import FastAPI / import sqlite3	使用别人写好的库前先导入
定义函数与类型注解	def get_car(db_path, car_id) -> Car:	-> Car 表示返回值是 Car 类型，只是提示，不强制
默认参数	def get_db_path() -> str:	-> str 表示返回字符串
f-string 格式化	f"/api/cars/{car.id}"	字符串前加 f，{} 内可以放变量或表达式
装饰器	@app.get("/cars")	给函数加功能：把普通函数注册成 Web 路由
类与继承	class Car(CarInput): id: int	Car 继承 CarInput 的所有字段，再加一个 id
assert 断言	assert car.make == "Mazda"	测试的核心：断言不成立就报错，测试即失败
with 上下文管理	with _connect(db_path) as conn:	自动管理资源（如数据库连接），结束时自动关闭/提交
列表推导式	[_row_to_car(r) for r in rows]	对列表每个元素做转换，快速生成新列表

pytest 与 fixture（预习）

pytest 是 Python 最流行的测试框架，第四章会深入。两个先验概念：

- 测试函数：名字以 test_ 开头的函数，里面用 assert 检查预期。pytest 会自动发现并运行所有 test_ 函数。

- fixture（夹具）：测试运行前自动准备的“道具”。例如课程项目的 database_path fixture 会为每个测试准备一个全新的临时数据库文件，保证测试之间互不影响、也不碰真实数据。

```
import pytest

@pytest.fixture
def database_path(tmp_path):
    return str(tmp_path / "test.db")   # tmp_path 是 pytest 内置的临时目录

# 测试函数把 fixture 名字写在参数里，pytest 自动注入
def test_saved_car_can_be_read_from_sqlite(database_path):
    ...
```

2.3 深入：装饰器（@ 到底是什么）

装饰器是 Python 中无处不在的语法，课程项目里的 @app.get("/cars")、@pytest.fixture 都是它。本节拆到最细，一步步看清楚每一行做了什么。

准备工作一：名字只是标签

理解装饰器前，必须先建立一个观念：函数名只是一个标签，贴在某个函数对象上；标签可以随时撕下来贴到别的东西上。

```
def say_hi():
    print("hi")

print(id(say_hi))   #  say_hi 标签当前贴在某个函数对象上

say_hi = 42         # 把标签撕下来，贴到整数 42 上
print(say_hi)       # 42—名字还在，但指向的东西换了
```

say_hi = 42 并没有“修改”原来的函数，只是 say_hi 这个名字不再指向它。装饰器做的事情，本质就是这样一次“重新贴标签”。

准备工作二：函数里面可以定义函数（闭包）

装饰器依赖两个语法事实：函数内部可以定义函数；内部函数可以记住外层函数的变量（这叫闭包）：

```
def outer():
    message = "外层变量"
    def inner():              # 函数里定义函数，合法
        print(message)        # inner 记住了外层的 message
    return inner              # 把内部函数作为返回值交出去

f = outer()    # outer 执行完，message 本该消失……
f()            # 但 inner 里还能打印出来："外层变量"
```

关键点：outer() 执行结束后，它内部的变量 message 按理应该被回收，但 inner 函数还“握着”它，所以依然能访问。这个“内部函数 + 它记住的外层变量”的组合就叫闭包。装饰器里的 wrapper 正是靠闭包记住原函数的。

手动装饰：一步一步拆解

先不看 @，看最原始的写法。完整代码如下，后面逐行拆：

```
def my_decorator(func):
    def wrapper():
        print("调用前做点事")
        result = func()
        print("调用后做点事")
        return result
    return wrapper

def say_hi():
    print("hi")

say_hi = my_decorator(say_hi)
say_hi()
```

这一行是全篇最重要的一行：

```
say_hi = my_decorator(say_hi)
```

把它拆成四步看：

1. 把原来的 say_hi 函数对象传给 my_decorator，即执行 my_decorator(say_hi)。
2. my_decorator 内部，参数 func 接住了这个函数对象（func 现在就是原来的 say_hi）。
3. my_decorator 执行 return wrapper，返回内部定义的 wrapper 函数对象。
4. 赋值号把标签 say_hi 从原函数上撕下来，贴到 wrapper 上——say_hi 这个名字从此指向 wrapper。

注意：执行 say_hi = my_decorator(say_hi) 这一行时，wrapper 里面的代码一行都不会运行。它只是被定义、然后被返回。真正打印“调用前做点事”发生在后面执行 say_hi() 时。

那么 say_hi() 时发生了什么？此时 say_hi 指向 wrapper，所以实际执行的是 wrapper()：

1. wrapper 打印“调用前做点事”。
2. wrapper 执行 func()——func 是闭包记住的原函数，也就是真正的 say_hi——打印“hi”。
3. wrapper 打印“调用后做点事”，返回结果。

所以输出是：

```
调用前做点事
hi
调用后做点事
```

这就是“手动装饰”：不改动原函数的一行代码，却给它包上了一层前后逻辑。

第二步：@ 语法糖只是自动帮你写那一行

每次手动写 say_hi = my_decorator(say_hi) 太啰嗦，Python 提供 @ 写法：

```
@my_decorator
def say_hi():
    print("hi")
```

这两段代码完全等价：

- 上方：定义 say_hi，然后自动执行 say_hi = my_decorator(say_hi)。
- 下方：先定义 say_hi，再手动写 say_hi = my_decorator(say_hi)。

看到 @xxx 时，脑子里自动翻译成一句话：“定义完这个函数后，把它交给 xxx，并用返回的东西替换这个名字。”被替换的函数名不变，指向的已经是加强版。

> **高频误区**
> @my_decorator 不是“调用 say_hi”，而是“换掉 say_hi”。
> 换标签的那一刻，wrapper 内部代码不执行；直到有人调用被装饰的函数，wrapper 才运行。

能处理任意参数的标准写法：再拆一遍

上面的 wrapper() 不接受参数，只能装饰无参函数。真实的被装饰函数几乎都有参数和返回值，标准写法如下：

```
import functools

def my_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"即将调用 {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def add(a, b):
    return a + b

print(add(2, 3))
```

逐点拆解：

1. @my_decorator 等价于 add = my_decorator(add)：定义完 add 后立刻换标签，此后 add 指向 wrapper。
2. *args 接收所有位置参数：调用 wrapper(2, 3) 时，args 是元组 (2, 3)；**kwargs 接收所有关键字参数：wrapper(a=2, b=3) 时，kwargs 是字典 {"a": 2, "b": 3}。两个合起来表示“不管什么参数都照单全收”。
3. func(*args, **kwargs) 是把收进来的参数原样拆开传给原函数：args=(2, 3) 拆成 func(2, 3)。参数进来什么样，传下去还是什么样。
4. @functools.wraps(func) 把原函数的名字、文档等元数据复制到 wrapper 上。不加它的话，add.__name__ 会变成 "wrapper" 而不是 "add"——打印“即将调用 wrapper”会让人困惑；加了就仍然是“即将调用 add”。

执行 print(add(2, 3)) 的完整流程：

1. add 已经指向 wrapper，所以 add(2, 3) 实际是 wrapper(2, 3)：args = (2, 3)。
2. wrapper 打印“即将调用 add”。
3. wrapper 执行 func(2, 3)——func 是闭包记住的原 add——得到 5。
4. wrapper 用 return 把 5 交出去。
5. 外层的 print 打印 5。

最终输出：

```
即将调用 add
5
```

一句话总结标准装饰器四件套：@ 语法糖换标签；*args/**kwargs 兼容任意参数；functools.wraps 保留原函数身份；return func(...) 把结果原样交回。不修改原函数，只在外面加功能。

带参数的装饰器（看懂即可）

课程里的 @app.get("/cars") 比上面多一层：装饰器自己还带参数。它的内部是三层嵌套：最外层函数接收 "/cars"，返回一个真正的装饰器，再由它去包装视图函数。你不需要会写，见到时知道“@ 后面跟括号 = 装饰器带配置参数”即可。

先补一课：“.” 号连接的是什么

functools 是 Python 自带的一个模块（可以粗浅理解为“库”，本质就是一个 .py 文件），wraps 是定义在这个文件里的一个函数；functools.wraps 就是“functools 模块里的 wraps 函数”。

点号在 Python 里是通用的“到里面找”的意思：点左边是一个对象，点右边是它内部的一项。模块是对象，模块里定义的函数就是它的成员。除了模块，别的对象也能用点号：

```
import os
os.path.join("a", "b")   # os 模块 → path 子模块 → join 函数，一层层往里找
"abc".upper()               # 字符串对象 → 它的 upper 方法
car.model                   # car 对象 → 它的 model 属性
```

所以 @functools.wraps(func) 里的点号跟装饰器机制本身无关，只是在说明“wraps 这个工具来自 functools 工具箱”。同一个工具，import 写法不同，点的写法也不同：

```
import functools
@functools.wraps(func)   # 拿到整个工具箱，用时写全名

from functools import wraps
@wraps(func)             # 只把 wraps 这一个工具拿进来，直接用短名
```

两种写法完全等价，选哪种只看哪种更清楚。课程里还能见到更长的链条：pytest.mark.parametrize 是“pytest 包 → mark → parametrize 函数”，os.environ.get 是“os 模块 → environ 字典 → 它的 get 方法”。读法都一样：从左到右，一层层往里进。

functools.wraps 内部到底包装了什么

先解开一个容易困惑的结构问题：@functools.wraps(func) 为什么带括号？因为 wraps 本身是一个“装饰器工厂”——它先接收原函数 func，返回一个真正的装饰器，再由这个装饰器去包装 wrapper。这和带参数的装饰器（如 @app.get("/cars")）是同一套路：带括号 = 先配置、再装饰。

它做的事，手工写出来大致如下（简化版，名字就叫 wraps）：

```
def wraps(func):
    def decorator(wrapper):
        wrapper.__name__ = func.__name__        # 把原函数的名字贴到 wrapper 上
        wrapper.__doc__ = func.__doc__          # 把原函数的说明文字贴过去
        wrapper.__module__ = func.__module__    # 把原函数所属模块名贴过去
        wrapper.__dict__.update(func.__dict__) # 把原函数自己挂的额外属性合并过去
        wrapper.__wrapped__ = func              # 留一条线索：wrapper 里能找到原函数
        return wrapper
    return decorator
```

逐点拆解：

1. “贴标签”再次发生：wrapper.__name__ = func.__name__ 这一行，右边先读 func 的出生名（"add"），左边把 wrapper 的同名属性改掉——wrapper 从此对外“冒名”成了 add。
2. 真实标准库的实现比这多一点（内部用了一个名单统一处理 __name__、__qualname__、__doc__ 等），但核心就是上面这几行赋值，没有任何魔法。
3. 亲眼看差异：删掉 @functools.wraps(func) 后 print(add.__name__) 显示 "wrapper"；加回去再运行，显示 "add"。print(add.__wrapped__(2, 3)) 甚至能绕过 wrapper 直接调用原函数。

为什么要加它？因为很多机制靠 __name__ 辨认函数：日志和报错显示“哪个函数出事了”、FastAPI 按函数名生成接口文档、pytest 按函数名收集和显示测试。如果所有被装饰函数的名字都变成 wrapper，这些信息就全乱了套。

“__name__”这种两头双下划线的名字是什么意思

这种写法有个外号叫 dunder（double underscore 的缩写），__name__ 读作“dunder-name”。它是 Python 的约定：凡是用 __ 两头包住的名字，都是 Python 预留给“内部协议或元数据”的特殊成员。课程里会碰到的几个：

写法	属于谁	含义
func.__name__	函数、类、模块	定义时使用的名字（出生名）
func.__doc__	函数、类、模块	文档字符串：写在定义第一行的说明文字
cls.__init__()	类	构造对象时自动调用的方法（第二章类与对象会讲）
module.__file__	模块	这个模块对应的文件路径
"__main__"	（作为字符串值使用）	一个 .py 被直接运行时，它的 __name__ 的值

判别“这是谁的 name”只有一条规则：属性永远属于点号左边的对象。add.__name__ 读的是 add 背后的函数对象上记录的出生名；写成 wrapper.__name__ 读的就是 wrapper 的出生名。赋值号两边各归各：wrapper.__name__ = func.__name__ 是“读 func 的、写 wrapper 的”，互不冲突。

课程代码里常见的 if __name__ == "__main__": 也是同一机制：一个 .py 文件被直接运行时，Python 把它的 __name__ 赋成字符串 "__main__"；被别的文件 import 时，__name__ 是模块名。所以这行判断的意思是：“只有直接运行本文件，才执行下面这段”（被 import 时不执行）。

最后一个提醒：双下划线名字是 Python 的保留地盘，自己给变量起名时不要发明 __xxx__ 式的名字，避免和未来的 Python 版本撞车。

课程中的三个真实装饰器

写法	它在做什么
@pytest.fixture	告诉 pytest：这个函数是“测试道具”，测试用例把它的名字写进参数就能获得道具（临时数据库、测试客户端等）

写法	它在做什么
@app.get("/cars")	告诉 FastAPI：这个函数负责处理“GET /cars”请求，请把它登记进路由表
@app.post("/api/cars")	同上，登记“POST /api/cars”路由；@app.put、@app.delete 同理

它们和上面例子的区别只在于“换标签时额外登记了什么”：fixture 被记入道具库，视图函数被记入路由表。机制一模一样。

> **动手练一次**
> 写一个 timing 装饰器：调用前用 time.time() 记录时间，调用后打印耗时。套在任意函数上跑通，你就彻底理解装饰器了。
> 检验是否真懂：说出 @timing 之后、真正调用被装饰函数之前，timing 内部的 wrapper 执行过吗？（答案：没有，只是换标签。）

pytest 的装饰器内部做了什么（@pytest.fixture / @pytest.mark.parametrize）

@pytest.fixture 的“包装”不是换标签，而是登记。拆开看一共五步：

1. 打标记：被 @pytest.fixture 装饰的函数会被贴上内部标记，登记进 pytest 的一张“道具登记表”，键就是函数名（如 database_path）。函数名本身不变，也没有 wrapper 包它。
2. 扫参数：pytest 收集到一个测试函数后，查看它的参数列表。例如 def test_x(database_path): 的参数里有 database_path，就去登记表查有没有这件道具。
3. 注入：查到就调用这个 fixture 函数，把返回值当作 database_path 参数传进测试函数。所以“把 fixture 名字写进参数”就等于“申请使用这件道具”，名字对上是关键。
4. 排链条：fixture 的参数也可以是别的 fixture 的名字——课程项目的 database_path(tmp_path) 里，tmp_path 是 pytest 内置的 fixture（每次提供一个全新临时目录）。pytest 按依赖关系自动排好调用顺序：先内层、后外层。
5. 做善后：fixture 里可以写 yield：yield 之前是准备工作（建临时数据库），yield 把道具交出去，测试跑完后继续执行 yield 之后的清理工作（删临时文件，不留垃圾）。

@pytest.mark.parametrize("year", [1800, 2020]) 是另一种“复制”：它把一个测试函数按参数列表复制成好几份，每份代入一个值分别运行，互不影响。所以参数化测试里只有一组失败时，报告只标出那一组。

回到 FastAPI 的 @app.get("/cars")：app 是 FastAPI() 创建出来的应用对象，get 是它上面的方法。执行到 @app.get("/cars") 这一行时，FastAPI 把“GET /cars → 这个函数”登记进 app 内部的路由表（函数名同样不变）。之后浏览器请求 GET /cars，它查表找到函数、执行、把返回值回给对方。装饰器在这里的全部工作就是这一次“登记”。

> **动手练一次**
> 删掉 @functools.wraps(func) 后 print(add.__name__)，再加回来 print 一次，亲眼看到差异；然后 print(add.__wrapped__(2, 3)) 直接调用原函数。
> 写两个 fixture：一个返回列表 [1, 2, 3]，一个依赖它并返回元素个数；写一个测试函数把两

个名字都写进参数，验证注入的值和顺序。

2.4 深入：类与对象

类（class）是“图纸”，对象（object）是按图纸造出来的“实物”。类把数据（属性）和操作数据的方法（函数）捆在一起。schemas.py 里的 CarInput、Car 就是类。

最小例子：定义一个类

```
class Car:
    def __init__(self, make, model, year):   # 构造方法：创建对象时自动调用
        self.make = make      # self.属性 = 值，把数据存到对象自己身上
        self.model = model
        self.year = year

    def description(self):                    # 方法：第一个参数永远是 self
        return f"{self.make} {self.model} ({self.year})"

car = Car("Toyota", "Yaris", 2020)   # 创建对象（实例化）
print(car.make)            # 访问属性：Toyota
print(car.description())   # 调用方法：Toyota Yaris (2020)
```

- __init__ 是构造方法：Car("Toyota", ...) 时自动执行，self 指刚创建出来的那个对象本身。
- self.make = make 把参数存成对象的属性；不同对象的属性互不影响。
- 方法的第一个参数固定叫 self（名字可以换，但大家都这么写），调用时不用传，Python 自动把对象传进去。

继承：在已有类的基础上扩展

```
class CarInput:            # 父类：四个字段
    def __init__(self, make, model, year, color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color

class Car(CarInput):       # 子类：继承父类的全部内容
    def __init__(self, id, make, model, year, color):
        super().__init__(make, model, year, color)   # 先让父类干它的活
        self.id = id         # 再补充自己的新属性

car = Car(1, "Toyota", "Yaris", 2020, "Blue")
print(car.color)           # 来自父类：Blue
print(car.id)              # 来自子类：1
```

对照课程项目 schemas.py：class Car(CarInput) 表示 Car 继承 CarInput 的四个字段，再加一个 id。继承的意义是“复用 + 特化”：子类自动拥有父类的一切，只写增量部分。

Pydantic：用“声明”代替“手工”

前面手写的类有三个麻烦：每个字段都要在 __init__ 里重复写一遍赋值（啰嗦）；传错类型不报错（无校验）；year 传字符串 "2020" 也不能自动变整数（无转换）。Pydantic 把这三个麻烦一次性解决，思路叫“声明式”：你只写“数据长什么样”，构造、校验、转换全部自动执行。

先看课程项目 schemas.py 的完整代码，再逐行拆：

```
from typing import Annotated
from pydantic import BaseModel, Field, StringConstraints

NonBlank = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1)]

class CarInput(BaseModel):
    make: NonBlank
    model: NonBlank
    year: int = Field(ge=1886, description="Model year")
    color: NonBlank

class Car(CarInput):
    id: int
```

逐行拆解：

1. class CarInput(BaseModel)：声明“这是一个数据类，交给 BaseModel 管理”。BaseModel 会在背后替你生成 __init__、校验逻辑、类型转换等一整套方法——所以文件里一个 __init__ 都看不到。
2. make: NonBlank：这一行声明了四件事——属性名 make；类型是 str；自动去除首尾空格（strip_whitespace=True）；不能为空（min_length=1）。它只是“规则声明”，不是赋值语句。
3. year: int = Field(ge=1886)：属性 year 是整数，且必须大于等于 1886。Field(...) 用来附加单靠类型表达不了的约束；description 只是给人看的说明文字。
4. class Car(CarInput)：继承（见上一小节）。Car 自动拥有父类全部四个字段，再补充一个自己的 id: int。

创建对象时，Pydantic 内部按什么顺序干活

执行 car = CarInput(make=" Mazda ", model="MX-5", year="2020", color="Red") 时，BaseModel 自动生成的 __init__ 按四步走：

1. 收集：把关键字参数按属性名对号入座（make 拿到 " Mazda "，year 拿到 "2020"……）。
2. 转换：按声明的类型把值转成目标类型。year 声明为 int，字符串 "2020" 被自动转成整数 2020；转换失败（如 year="abc"）在这一步就报错。
3. 校验：按声明的约束逐项检查。" Mazda " 去空格后非空，通过；假如 year=1800，ge=1886 不满足，在这一步抛出 ValidationError。
4. 赋值：全部通过后，才把值存进对象的属性。

不合法数据的两种失败现场：

```
# 转换失败：TypeError 风格的报错
car = CarInput(make="Mazda", model="MX-5", year="abc", color="Red")

# 校验失败：抛出 pydantic.ValidationError，里面详细列出每条违规
car = CarInput(make="Mazda", model="MX-5", year=1800, color="Red")
# 错误信息大意：
# Input should be greater than or equal to 1886  （输入应大于等于 1886）
```

注意一个容易混淆的点：min_length=1 校验的是“去空格之后”的字符串。所以 model=" "（一

个空格）会先被 strip 成空串，再因长度 0 触发 ValidationError——这正是第四章单元测试 test_blank_model_is_rejected 在验证的行为。

model_dump()：对象转字典

Pydantic 对象不能直接被 JSON 序列化（JSON 只认字典、列表、数字、字符串），所以 BaseModel 提供了 model_dump() 把对象转成普通字典：

```
car = CarInput(make="Mazda", model="MX-5", year=2020, color="Red")
print(car.model_dump())
# {'make': 'Mazda', 'model': 'MX-5', 'year': 2020, 'color': 'Red'}

# FastAPI 返回 JSON 时内部就做了这一步（main.py 里的 content=car.model_dump()）
```

回到 FastAPI：一个声明在三个地方生效

为什么课程项目敢把全部输入校验寄托在这 19 行上？因为同一份声明在整条链路上反复生效：

1. 单元测试：直接构造 CarInput(...)，验证转换和校验行为（第四章第 1 层）。
2. API 处理：路由参数写成 payload: CarInput 时，FastAPI 把请求体 JSON 自动交给 CarInput 走“收集→转换→校验”四步；任何一步失败直接返回 422，视图函数一行都不会执行。
3. 数据落库：校验通过的对象传给 database.py，取出的字段必然符合声明，数据库写入不需要再校验一次。

> **动手练一次**
>
> 新建一个文件，把上面的 CarInput 原样敲一遍（不要复制），然后依次执行：合法构造、year 传 "2020" 字符串、year 传 1800、model 传 " "——每次观察报不报错、报什么错。
>
> 检验是否真懂：CarInput(make="Mazda", model="MX-5", year="1800", color="Red") 会成功还是失败？（答案：失败——字符串 "1800" 先被转换成整数 1800，随后在校验步被 ge=1886 拦下。注意它失败在校验那一步而不是转换那一步，想明白这个次序，四步流程就真懂了。）

类变量与实例变量（了解）

- self.xxx = ... 定义的是实例变量，每个对象一份；在方法外直接写的 xxx = ... 是类变量，全体对象共享一份。
- 其他语言中的“私有属性”在 Python 里靠约定实现：名字前加一条下划线 _internal 表示“内部使用，请勿动”。

> **动手练一次**
>
> 自己写一个 Dog 类：属性 name、age，方法 bark() 返回 "汪汪"。再造一个子类 GuideDog，多一个属性 owner，bark() 返回 "汪汪，我是导盲犬"。
>
> 对照读 schemas.py：找出 BaseModel、继承（Car 继承 CarInput）、Field 约束分别在哪儿。

2.5 HTTP 基础：请求方法、状态码与常见名词

cars-crud 是一个 Web 应用：浏览器访问网址，服务器返回页面或数据。理解几个概念就能读懂第四章。

HTTP 请求方法（动词）

方法	含义	cars-crud 中的例子
GET	读取数据	GET /cars 显示列表页；GET /api/cars 返回 JSON 列表
POST	新建数据	POST /api/cars 创建一辆车
PUT	更新数据	PUT /api/cars/{id} 更新一辆车
DELETE	删除数据	DELETE /api/cars/{id} 删除一辆车

常见状态码

状态码	含义	什么时候见到
200	成功	GET/PUT 请求正常返回
201	已创建	POST 创建资源成功（cars-crud 创建车辆返回 201）
302/303	重定向	表单提交后跳转到详情页
404	找不到	请求不存在的车辆
422	参数校验失败	提交的年份 1800 不合法，被 Pydantic 拦截

其他名词

- URL：网址，如 http://127.0.0.1:8000/cars/new。127.0.0.1 表示“本机”，8000 是端口号。
- JSON：一种数据格式，长得像 Python 字典。前后端、API 测试都用它传数据。
- API：应用对外提供的数据接口。cars-crud 的 /api/cars 系列就是 JSON API，而 /cars 系列返回 HTML 页面。
- 前端页面里的 fetch()：创建车辆页面的 JavaScript 用 fetch 向 /api/cars 发送 POST 请求——这就是“浏览器 → JS → FastAPI → SQLite”完整链路的浏览器端。

2.6 Python Web 开发入门

前面 2.5 节讲了 HTTP 协议本身（请求方法、状态码），本节回答另一个问题：用 Python 写一个网站，代码到底怎么组织？读完本节再回头看 cars-crud 的 main.py，每一行都会变得亲切。

Web 应用的本质：一台永远等着回话的机器

Web 应用就是一个常驻内存的程序，循环做三件事：

1. 监听某个端口（如 8000），等待浏览器发来 HTTP 请求。
2. 根据请求的内容（方法 + 路径）找到对应的处理函数，执行它。

3. 把处理结果（HTML 页面或 JSON 数据）装成 HTTP 响应返回给浏览器。

你不需要自己写这个循环——那是 Web 框架的活。Python 生态里 Django（大而全）、Flask（轻量）、FastAPI（现代、自带数据校验和 API 文档）是三大主流选择，本课程用 FastAPI。

最小的 FastAPI 应用：只有 5 行

```
from fastapi import FastAPI

app = FastAPI()                    # 创建一个应用对象

@app.get("/")                      # 注册路由：GET 请求 / 时执行下面的函数
def home():
    return {"message": "hello"}    # 返回字典，FastAPI 自动转成 JSON
```

启动它：

```
python -m uvicorn main:app --reload --port 8000
# 浏览器打开 http://127.0.0.1:8000  看到 {"message": "hello"}
# 打开 http://127.0.0.1:8000/docs   有自动生成的交互式 API 文档
```

这里出现了两个角色：FastAPI 负责“路由分发和数据校验”；uvicorn 是 ASGI 服务器（真正监听端口、收发 HTTP 消息的发动机）。开发时二者一起跑；Docker 部署时 CMD 里也是它们俩。

路由：把 URL 映射到函数

路由就是“URL 模式 → 处理函数”的登记表。@app.get("/cars/{car_id}") 这类写法中：

- HTTP 方法决定动作：GET 读、POST 建、PUT 改、DELETE 删（REST 风格）。
- 路径里的 {car_id} 是路径参数，FastAPI 自动提取并转成声明的类型（car_id: int 表示必须是整数，否则返回错误）。
- 查询参数如 /api/cars?page=2 对应函数参数 page: int = 1。
- POST/PUT 的请求体 JSON 自动转成 Pydantic 对象（如 payload: CarInput），不合法直接返回 422。

```
@app.get("/cars/{car_id}")             # 路径参数
def detail_page(car_id: int):           # : int 声明类型，自动转换与校验
    ...

@app.post("/api/cars")                  # 请求体校验
def create(payload: CarInput):          # JSON 自动转 CarInput 对象
    ...
```

响应：函数 return 的东西就是浏览器收到的东西

返回内容	浏览器收到
return 字典或列表	JSON 数据（API 的常见形态）
return HTMLResponse(" ... ")	HTML 页面
return RedirectResponse("/cars")	重定向指令（浏览器自动跳转）
raise HTTPException(status_code=404)	错误响应，状态码由你指定

依赖注入：FastAPI 最优雅的设计

cars-crud 的几乎每个路由都有 db: str = Depends(get_db)。它的含义是："执行这个路由前，先调用 get_db()，把它的返回值传给我"。好处：

- 每个请求都会拿到一个初始化好的数据库路径，路由函数不用关心初始化细节。
- 测试时可以"偷梁换柱"：测试把 CARS_DB_PATH 环境变量指向临时数据库，get_db() 返回的就是临时库——这就是为什么测试不碰真实 cars.db。
- Depends 还能嵌套（Depends 里再 Depends），适合组装复杂依赖（数据库连接、当前用户等）。

一个请求在 cars-crud 里的完整旅程

```
浏览器 GET /cars/3
   |
   ▼
uvicorn（监听 8000 端口，解析 HTTP 报文）
   |
   ▼
FastAPI 路由表匹配：GET /cars/{car_id} → detail_page 函数
   |
   ▼
依赖注入：先执行 get_db() 得到数据库路径 db
   |
   ▼
执行 detail_page(3, db)：调用 database.get_car(db, 3)
   |
   ▼
没找到 → raise HTTPException(404)；找到 → 拼 HTML 字符串返回
   |
   ▼
uvicorn 把结果装成 HTTP 响应发回浏览器
```

前端页面从哪来：三种主流方案

方案	做法	课程项目用了吗
后端渲染	后端用模板把数据和 HTML 拼好再返回	是：main.py 里 _page()、_list_page() 等函数直接在 Python 里拼 HTML 字符串
静态站点 + API	后端只提供 JSON API，前端是独立的 HTML/JS 页面	部分是：/cars/new 页面的表单由 JavaScript 用 fetch 调用 /api/cars
前端框架	React/Vue 等单页应用，通过 API 与后端通信	否（课程为聚焦后端刻意保持简单）

cars-crud 把页面 HTML 直接写在 Python 字符串里（配合 app/static 目录放 CSS），是为了让项目最小化、把注意力集中在测试和部署流程上。真实项目通常用模板引擎（Jinja2）或前后端分离。

动手练一次

从零写上面那个 5 行 FastAPI 应用并跑通，再加一个 GET /add?a=1&b=2 返回两数之和——你就掌握了路由、查询参数、JSON 响应三件套。

然后对照 main.py 找：路由注册、路径参数、请求体校验、依赖注入、HTML 响应、重定向、404，各出现一次，逐一指认。

2.7 YAML 基础（GitHub Actions 用）

GitHub Actions 的工作流文件是 YAML 格式。YAML 用缩进表达层级，用“键: 值”表达配置：

```
name: Cars CRUD Tests          # 键：值，# 后面是注释
on:                            # 顶层键
  pull_request:                # 缩进两格，属于 on
    branches: [main]           # 再缩进两格；[main] 是列表
jobs:                          # 顶层键，与 on 平级
  tests:                       # 任务名，可自定义
    runs-on: ubuntu-latest
```

易错点

YAML 只能用**空格**缩进，不能用 Tab；同一层级的缩进必须完全一致。

键名（name、on、jobs、runs-on、steps、uses、run、with）是预定义关键字，拼错一个字母配置就失效。

列表写成 [main] 或每行一个 - item 都可以，但不要混用导致层级错乱。

2.8 环境安装检查清单

1. Git：git --version 能看到版本号（课程要求 2.28 或更高）。
2. Python：python --version ≥ 3.11。
3. VS Code：任意文本编辑器均可，推荐 VS Code。
4. Docker Desktop（第六章才需要）：docker --version 和 docker info 都能正常输出。国内网络拉取镜像慢时，课程提供了 Azure 代理（第六章会讲）。
5. GitHub 账号：注册并确认能打开 github.com。

第三章 Git 与 GitHub 版本控制

3.1 为什么需要版本控制

版本控制解决三个问题（课堂笔记 L2 的三大核心能力）：

1. 历史回溯：随时回退到任意历史版本。某段代码出了问题，不用手动逐行删除，直接回到出错之前的状态。
2. 多人协作：自动整合多名开发者的工作，替代手动对比合并文件的低效方式。

3. 分支隔离：在不破坏稳定代码的前提下，多人分别在独立副本上开发新功能；成功则合并，失败则丢弃分支。

3.2 核心概念

概念	通俗解释
工作区（Working directory）	你磁盘上正在编辑的文件
暂存区（Staging area）	为下一次提交所选定的更改（git add 就是把改动放进暂存区）
提交（Commit）	带提交信息和唯一 ID 的代码快照，相当于游戏的“存档点”
分支（Branch）	指向某个提交的名字，在它上面提交时会向前移动；默认主分支叫 main
HEAD	指向你当前所在分支/提交的指针，git log --graph 里用 HEAD 标记你在哪
远程（Remote）	指向另一个仓库的连接，通常叫 origin；GitHub 上的仓库就是你的远程仓库
main 与 origin/main	前者是你本地的 main 分支，后者是你在本地记录的远程 main 的状态，由 fetch/pull 更新
拉取请求（Pull Request）	GitHub 上的评审流程：提议把一个分支合并进另一个分支，可附带讨论和自动检查。它不是 Git 命令

一个经典比喻：Git 把代码状态当游戏存档。每个 commit 是一个新存档，分支是从某个存档分叉出的平行时间线，合并是把两条时间线的成果合到一个新存档里。

3.3 常用命令速查

日常命令

命令	作用
git init -b main	在当前文件夹初始化仓库，主分支命名为 main
git clone <url>	把远程仓库下载到新文件夹
git remote add origin <url>	把本地仓库连接到名为 origin 的远程仓库
git status	查看当前分支与已暂存/未暂存/未跟踪的文件
git add <file>	把文件改动放入暂存区
git commit -m "信息"	把暂存区的改动保存为一个提交
git log --oneline	紧凑地查看提交历史；加 --graph --all 可看到分支图
git diff	查看未暂存的改动；git diff --staged 查看已暂存的改动
git switch -c feature	创建并切换到新分支；git switch main 切换回已有分支

命令	作用
git branch	列出本地分支
git merge feature	把 feature 分支合并进当前分支
git fetch origin	下载远程更新，但不合并进当前分支
git pull	下载并合并远程更新（= fetch + merge，行为可用参数控制）
git push / git push -u origin 分支	把本地提交上传到远程；-u 同时设置上游分支

容易混淆的三组命令

对比	关键区别
Commit vs. Push	commit 保存在本地；push 才把提交共享到远程
Merge vs. Rebase	merge 保留现有提交、可能产生合并提交；rebase 把提交重放到新基线上，产生线性历史但会改写提交 ID
Revert vs. Reset	revert 用一个新的提交来撤销历史（安全，已发布的提交用它）；reset 直接移动分支指针（只用于未发布的提交）
Fetch vs. Pull	fetch 只更新远程跟踪信息；pull 还会把更新合并进当前分支
Stash	把未提交的改动先搁到一边（git stash），之后用 git stash pop 取回

3.4 配套练习逐步讲解（git-guide-and-practice 的 15 个练习）

练习材料：练习指南/中文版/1. git-guide-and-practice.pdf。建议在 git-playground/ 目录里边看本文档边操作。每个练习都遵循“执行命令 → 观察结果 → 对照检查点”的节奏。文中 YOUR_USERNAME 一律替换成你的 GitHub 用户名。

> **开始前**
> 全程在同一个终端里操作，除非练习明确要求切换目录。
> 如果 Git 打开了分页器（日志太长时），按 q 退出。
> 标注“预期失败”的步骤失败是正常现象，按说明继续即可。

练习 1-2：创建空仓库并把本地文件夹连接到 GitHub

第一部分在 GitHub 网页操作：New repository → 命名 git-playground → 不勾选 README/.gitignore/license（保持空仓库）→ 复制 HTTPS 地址。第二部分在本地：

```
mkdir git-playground
cd git-playground
git init -b main
git config user.name "Your Name"
git config user.email "YOUR_EMAIL"
```

```
# 创建三个练习文件
echo "My Git practice notes" > notes.txt
echo "Hello world" > greeting.txt
echo "color=blue" > settings.txt

# 查看状态 → 暂存 → 查看暂存差异 → 提交
git status
git add notes.txt greeting.txt settings.txt
git diff --staged
git commit -m "Create practice files"

# 连接远程并推送
git remote add origin https://github.com/YOUR_USERNAME/git-playground.git
git remote -v
git push -u origin main
```

逐段解释：git init 生成隐藏的 .git 文件夹（存全部历史）；git add 把文件放进暂存区；git commit 生成第一个存档；git remote add 建立本地与 GitHub 的关联；-u 让以后的 git push/git pull 不用再加参数。user.name/user.email 只标识提交者身份，不是登录认证——HTTPS 推送时的密码要用凭据管理器或个人访问令牌，账户密码无效。

检查点：刷新 GitHub 页面，三个文本文件都出现在 main 分支上。

练习 3：检查、暂存、提交并推送一次编辑

```
echo "I can inspect and save changes." >> notes.txt
git status          # 显示 notes.txt 被修改
git diff            # 显示具体改了一行
git add notes.txt
git diff            # 现在空了：改动已进暂存区
git diff --staged   # 显示暂存中的改动
git commit -m "Add a practice note"
git log --oneline -3
git push
```

这一步教你用 status/diff 两个“观察哨”：改动先在工作区，add 之后进暂存区，commit 之后工作区变干净。只有 push 才更新 GitHub。

练习 4：创建分支并合并

```
git switch -c feature-list        # 创建并切换到新分支
echo "Practice branching" > tasks.txt
git add tasks.txt
git commit -m "Add practice tasks"
git branch                        # 确认当前在两个分支上各有一个提交
git switch main                   # 切回 main：tasks.txt 消失了！
ls
git merge --no-ff feature-list -m "Merge practice tasks"
cat tasks.txt                     # 文件回来了
git log --graph --oneline --all -8
```

直观体验分支隔离：切换到 main 时 feature 分支上的文件“消失”，合并后又出现。--no-ff 强制产生一个合并提交，让历史里能看出“这里发生过一次合并”。

练习 5：触发并解决合并冲突

冲突的产生：两个分支修改了同一文件的同一位置，Git 不知道该保留哪个，只能请你裁决。

```
git switch -c friendly-greeting
echo "Hello friends" > greeting.txt
git add greeting.txt
git commit -m "Use a friendly greeting"
git switch main
echo "Hello team" > greeting.txt
git add greeting.txt
git commit -m "Use a team greeting"
git merge friendly-greeting       # 预期失败：内容冲突
git status
cat greeting.txt
```

冲突文件里会出现冲突标记：<<<<<<< HEAD 到 ======= 之间是你当前分支（main）的版本，======= 到 >>>>>>> friendly-greeting 之间是对方分支的版本。解决冲突 = 编辑文件成最终想要的样子、删掉标记、add、commit。

```
# 先练习取消合并
git merge --abort
cat greeting.txt                  # 回到合并前的状态

# 再次触发冲突并真正解决
git merge friendly-greeting       # 预期再次失败
echo "Hello team and friends" > greeting.txt   # 覆盖掉冲突标记
git add greeting.txt
git commit -m "Resolve greeting conflict"
git status                        # 检查点：工作区干净，合并完成
```

易错点

看到冲突标记不要慌：它只是在“请示”你。要么手动编辑保留想要的内容，要么 git merge --abort 整体取消。

解决冲突后必须 git add + git commit 才算完成；只改文件不提交，冲突依然存在。

提交信息写 "Resolve greeting conflict" 或 "merge conflict resolution"，团队里一眼能看出这是冲突解决提交。

练习 6：触发并解决 rebase 冲突（选学，课堂建议学到第 6 页即可）

```
git switch -c color-update
echo "color=green" > settings.txt
git add settings.txt
git commit -m "Choose green"
git switch main
echo "color=red" > settings.txt
git add settings.txt
git commit -m "Choose red"
git switch color-update
git rebase main                   # 预期失败：无法自动重放提交

# 查看、取消、重试
cat settings.txt
git rebase --abort
git rebase main                   # 预期再次失败

# 用第三种颜色解决
echo "color=purple" > settings.txt
git add settings.txt
git -c core.editor=true rebase --continue
git log --oneline -3
```

rebase 与 merge 的本质区别：merge 把两条历史“汇合”，rebase 把你的提交“搬到”新基线上——因此提交 ID 会变（历史被改写）。所以课程反复强调：不要对已经推送到远程、别人在用的提交做 rebase/reset。git -c core.editor=true rebase --continue 的作用是沿用原提交信息，不打开编辑器。

练习 7：创建并合并一个 Pull Request

```
git switch -c pr-notes
echo "This file arrived through a pull request." > review.txt
git add review.txt
git commit -m "Add review notes"
git push -u origin pr-notes
```

然后在 GitHub 网页操作：Pull requests → New pull request → base 选 main、compare 选 pr-notes → 查看 Files changed → 选择 Create a merge commit → 合并并确认。检查点：GitHub 的 main 上有了 review.txt，但你的本地 main 还没有——这正是练习 8 要解决的问题。

练习 8：fetch 与 pull 的区别

```
git switch main
git fetch origin                                # 只下载远程信息
git log --oneline main..origin/main             # 远程有而本地没有的提交
git diff main origin/main
ls                                              # review.txt 仍不在本地！

git pull --ff-only                              # 现在才真正合并进来
cat review.txt                                  # 文件出现了
git branch -d pr-notes                          # 删除已合并的本地分支
```

main..origin/main 表示“可从 origin/main 到达、但不能从 main 到达”的提交集合。fetch 让你先看清远程发生了什么，pull 才动手合并。

练习 9：克隆第二个副本，模拟队友协作

这个练习用两个本地文件夹模拟两台电脑上的两名开发者，是理解“历史分叉”最关键的一步。

```
# 新建一个“队友”的克隆
cd ..
git clone https://github.com/YOUR_USERNAME/git-playground.git git-playground-peer
cd git-playground-peer
git config user.name "Your Name"
git config user.email "YOUR_EMAIL"
echo "A teammate added this note." > team.txt
git add team.txt
git commit -m "Add teammate note"
git push

# 回到自己的目录，不拉取就直接提交并推送
cd ../git-playground
echo "My simultaneous local change." >> notes.txt
git add notes.txt
git commit -m "Add local work"
git push                            # 预期失败：远程有你缺少的提交

# 观察分叉，再整合
```

```
git fetch origin
git log --oneline --left-right main...origin/main
git pull --ff-only                # 预期失败：分叉无法快进
git pull --rebase                 # 把你的提交重放到远程提交之后
git log --oneline -4
git push                          # 检查点：两边都保留，推送成功
```

三个点（main...origin/main）配合 --left-right 显示分叉两侧的提交：< 是仅本地的，> 是仅远程的。pull --rebase 是处理个人未发布提交的推荐姿势：先把远程的工作拿下来，再把自己的提交叠上去，历史保持线性。

练习 10：pull 的另一种姿势——合并

```
cd ../git-playground-peer
git pull --ff-only
echo "Another teammate change." >> team.txt
git add team.txt
git commit -m "Extend teammate note"
git push
cd ../git-playground
echo "Another local change." >> notes.txt
git add notes.txt
git commit -m "Extend local notes"
git pull --no-rebase --no-edit    # 用合并方式整合
git log --graph --oneline -6      # 检查点：出现了一个合并提交
git push
```

对比练习 9：这次 Git 自动创建了一个合并提交把两条历史连起来，你本地的提交 ID 保持不变。--no-edit 表示接受 Git 自动生成的合并信息。日常工作里 pull --rebase 和 pull --no-rebase 都常见，团队约定优先。

练习 11：stash 搁置未完成的工作

```
echo "An unfinished idea." >> notes.txt
git diff
git stash push -m "Unfinished notes"   # 把改动收起来，工作区变干净
git status
git stash list                         # 查看 stash 列表
cat notes.txt                          # 想法暂时消失了
git switch feature-list
git switch main
git stash pop                          # 取回改动
git diff
git add notes.txt
git commit -m "Finish the stashed idea"
```

典型场景：改到一半需要切分支处理急事，但不想为一个半成品提交。stash 就是“临时寄存”。普通 stash 不含未跟踪的新文件，加 -u 才包含；pop 时如果目标位置也被改过可能冲突，此时 stash 会保留、不会丢失。

练习 12：用 revert 撤销已发布的提交

```
echo "This file was a mistake." > mistake.txt
git add mistake.txt
git commit -m "Add an unwanted file"
git push
git revert --no-edit HEAD        # 创建一个新提交来“反做”上一个提交
git log --oneline -3             # 历史里两个提交都在
```

```
git push
```

重点：revert 不改写历史，而是在历史末尾追加一个“反向提交”。已经 push 出去的提交只能用这种方式撤销——这是对协作者的基本礼貌。

练习 13：用 reset 重做未发布的提交

```
git switch -c reset-practice
echo "Draft wording" > draft.txt
git add draft.txt
git commit -m "Temporary draft"
git reset --soft HEAD~1          # 回退一个提交，但保留暂存
git status
git diff --staged
git log --oneline -2

echo "Final wording" > draft.txt
git add draft.txt
git commit -m "Add the final draft"
git switch main
git merge --ff-only reset-practice
```

reset --soft 移动分支指针但保留改动（且保持已暂存）；--mixed 保留改动但不暂存（默认）；--hard 连工作区文件一起丢弃，危险。HEAD~1 指当前提交的第一个父提交。reset 只适用于还没推送的提交。

练习 14：取消暂存一个编辑，然后丢弃它

```
echo "An edit I do not want." >> notes.txt
git add notes.txt
git restore --staged notes.txt   # 取消暂存（改动还在文件里）
git status
git diff
git restore notes.txt            # 丢弃工作区的这次编辑
git status
```

restore --staged 把文件移出暂存区但保留修改；restore <file> 用暂存区（此时等于最后一次提交）的内容覆盖工作区，真正丢弃修改。注意：Git 无法恢复从未提交或 stash 的改动，discard 前三思。

练习 15：收尾

```
git push
git status
git log --graph --oneline --decorate --all -25
```

最终检查：你在 main 上、工作区干净、GitHub 有最终文件，历史里能看到分支提交、合并提交、rebase 重放和一次显式 revert——这正是一个小型团队协作历史的缩影。

3.5 冲突解决通用流程（背这个就够了）

1. 执行合并（git merge 分支），看到 conflict 提示。
2. git status 找到冲突文件，打开文件，找到 <<<<<<<、=======、>>>>>>> 标记。
3. 手动编辑成最终想要的内容，删除全部冲突标记。
4. git add 冲突文件，git commit 完成合并（信息写 "merge conflict resolution"）。

5. 如果中途想放弃：git merge --abort（rebase 冲突则用 git rebase --abort）。

3.6 常见错误速查

现象	原因与对策
push 被拒绝（rejected）	远程有你缺少的提交。先 git pull --rebase，解决冲突后再 push
自动合并失败，fix conflicts	同一位置被两边修改。按 3.5 节流程手动解决
git diff 看不到改动	改动已 add 进暂存区，改用 git diff --staged
切分支时提示会覆盖本地文件	先 commit 或 git stash，再切换
提示输入密码但账户密码无效	HTTPS 推送需要用个人访问令牌或凭据管理器，不是 GitHub 登录密码
不小心对公共分支做了 rebase/reset	立即停止并告知协作者；已推送的改写用 revert 补救，不要强推公共分支

第四章 软件测试与 FastAPI

4.1 为什么要做自动化测试

手工测试的困境：每新增一个功能，都要把旧功能全部重新手测一遍。项目小的时候无所谓，规模一大，工作量呈指数级增长，最终变得不可行。自动化测试的思路是：每写一个功能函数，就配套写一个测试函数，之后一条命令批量运行全部测试。测试用例只写一次，反复运行的边际成本几乎为零。

三大测试类型（测试金字塔）

类型	测什么	数量	课程中的例子
单元测试（Unit）	最小代码单元（单个函数/类），不碰数据库和网络	最多	CarInput 能否自动去除字符串首尾空格
集成测试（Integration）	多个组件协同工作，如应用代码 + 真实 SQLite	中等	create_car() 写入后能否用 get_car() 读回来
端到端测试（E2E）	模拟真实用户操作，覆盖浏览器到数据库全链路	最少	用 Playwright 驱动 Chromium 在页面创建一辆车

数量分布原则：单元 > 集成 > E2E。E2E 只覆盖核心主流程（1-2 条快乐路径加少量异常路径），细分场景交给前两层。

快乐路径与异常路径

- 快乐路径（Happy Path）：正常操作能走通，如合法邮箱通过校验。

- 异常路径（Sad Path）：非法输入被正确拦截，如年份 1800 被拒绝并返回 422，而不是崩溃。

测试驱动开发（TDD，了解即可）

1. 先梳理功能行为，编写全部测试用例。
2. 运行测试——此时全部失败，因为功能还没写。
3. 编写功能代码，直到所有测试通过。

好处：强制保证测试覆盖率，测试用例本身就是功能文档。课程练习不强制 TDD，但“写新功能必须配套写测试”是同一精神。

4.2 pytest 入门

pytest 是 Python 的测试框架。约定：测试文件名以 test_ 开头，测试函数以 test_ 开头，函数体内用 assert 断言预期。安装与运行：

```
pip install pytest          # 或 pip install -r requirements-dev.txt
python -m pytest -q           # 运行全部测试
python -m pytest tests/unit -q        # 只运行某个目录
python -m pytest tests/unit/test_workshop_unit.py -v  # 显示每个用例详情
```

输出里 PASSED 表示断言全部成立；FAILED 表示某个 assert 不匹配；ERROR 表示测试根本没跑起来（如缺依赖）。fixture 用 @pytest.fixture 定义，测试函数把 fixture 名写进参数即可自动获得（2.2 节已给示例）。

4.3 读懂 cars-crud 应用（逐文件讲解）

cars-crud 是一个“汽车信息增删改查”Web 应用：FastAPI 提供页面和 JSON API，SQLite 存数据，纯 HTML/CSS/JS 做前端。项目结构：

```
cars-crud/
|-- app/
|   |-- schemas.py      # Pydantic 输入校验（单元测试目标）
|   |-- database.py     # SQLite 操作（集成测试目标）
|   |-- main.py         # FastAPI 页面和 API 路由
|   `-- static/         # 原始 HTML、CSS、JavaScript（E2E 目标）
|-- tests/              # unit / integration / api / e2e 四个目录
|   `-- conftest.py    # database_path、client 等 fixture
|-- requirements.txt    # 运行依赖
|-- requirements-dev.txt# 运行 + 测试依赖
|-- pytest.ini
`-- Dockerfile / .github/workflows/   （第六、五章的对象）
```

schemas.py：Pydantic 数据校验

```
from typing import Annotated
from pydantic import BaseModel, Field, StringConstraints

# 一个“去掉首尾空格、且不能为空”的文本字段类型
NonBlank = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1)]

class CarInput(BaseModel):
```

```
    make: NonBlank     # 品牌
    model: NonBlank    # 型号
    year: int = Field(ge=1886, description="Model year")   # 年份必须 >= 1886
    color: NonBlank    # 颜色

class Car(CarInput):
    id: int            # 继承 CarInput 的四个字段，再加数据库主键 id
```

Pydantic 是 Python 的数据校验库：你用类声明“数据长什么样”，它自动做类型转换和校验。例：CarInput(make=" Mazda ", ...) 会自动把 " Mazda " 两端的空格去掉（strip_whitespace），这就是单元测试要验证的行为；年份小于 1886 会直接抛 ValidationError，这就是 API 测试里提交 1800 会返回 422 的原因。

database.py：SQLite 持久层

```
def create_car(db_path, car_input: CarInput) -> Car:
    initialize(db_path)          # 确保表存在
    with _connect(db_path) as conn:
        cur = conn.execute(
            "INSERT INTO cars (make, model, year, color) VALUES (?, ?, ?, ?)",
            (car_input.make, car_input.model, car_input.year, car_input.color),
        )
        car_id = cur.lastrowid   # 数据库自动分配的自增 id
    return get_car(db_path, car_id)
```

SQLite 是零配置的本地数据库，数据存在一个文件里（默认 cars.db）。核心函数五个：initialize（建表）、create_car、get_car、list_cars、update_car、delete_car。集成测试的目标就是验证“写入提交后，用另一个连接真的能读回来”。

main.py：FastAPI 路由与依赖注入

main.py 分两层：/cars 系列返回 HTML 页面，/api/cars 系列返回 JSON。看一个典型的 API 路由：

```
@app.post("/api/cars", status_code=201)
def create_car_api(payload: CarInput, db: str = Depends(get_db)):
    car = database.create_car(db, payload)
    return JSONResponse(
        status_code=201,
        content=car.model_dump(),                    # 把 Car 对象转成 JSON
        headers={"Location": f"/api/cars/{car.id}"},
    )
```

三个关键点：

- @app.post("/api/cars") 是装饰器，把这个普通函数注册为“处理 POST /api/cars 的路由”。
- 参数 payload: CarInput 是 FastAPI 的依赖注入：请求体 JSON 会被自动校验并转成 CarInput 对象，不合法就直接返回 422——业务代码一行校验都不用写。
- db: str = Depends(get_db) 也是依赖注入：每个请求都会调用 get_db() 拿到数据库路径，测试时可以用环境变量 CARS_DB_PATH 把它指向临时数据库。

创建页面（/cars/new）里有一段 JavaScript：表单提交时用 fetch('/api/cars', {method:'POST', ...}) 调用上面的 API——这就是 E2E 测试要驱动的完整链路：浏览器 → HTML 表单 → JS fetch → FastAPI → SQLite。

重点理解

三个文件各是一个测试层级的靶子：schemas.py ← 单元测试；database.py ← 集成测试；main.py 的 HTTP 接口 ← API 测试；整站 + 浏览器 ← E2E 测试。练习指南里的文件注释（unit-test target 等）说的正是这个对应关系。

每个测试都有独立临时数据库（conftest.py 里的 database_path fixture 用 pytest 的 tmp_path），所以测试不碰真实 cars.db，也不会互相影响。

4.4 四个测试层级实战（TESTING_WORKSHOP 练习）

练习材料：练习指南/中文版/2-2. TESTING_WORKSHOP.pdf。每个小节的节奏都一样：复制一个可用测试 → 运行看到通过 → 故意改错断言看到失败 → 恢复并自己写一个新测试。这个“复制→运行→改坏→修复→自创”的节奏是刻意设计的，让你对每个断言的作用建立直觉。

动手前：启动应用

```
# 终端 A：启动应用（保持运行）
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
# 浏览器打开 http://127.0.0.1:8000 手动创建/查看/编辑/删除一辆车
# 交互式 API 文档在 http://127.0.0.1:8000/docs，值得一逛
# 测试命令在另一个终端 B 执行；Ctrl+C 停止应用
```

第 1 层：单元测试（不碰数据库和网络）

目标：验证 Pydantic 的校验行为。示例测试：

```
from app.schemas import CarInput

def test_car_text_fields_are_trimmed():
    car = CarInput(make=" Mazda ", model=" MX-5 ", year=2020, color=" Red ")
    assert car.make == "Mazda"
    assert car.model == "MX-5"
    assert car.color == "Red"
python -m pytest tests/unit/test_workshop_unit.py -v   # 预期 1 passed
```

你的任务（test_blank_model_is_rejected）：构造 model=" " 的 CarInput，断言抛出 pydantic.ValidationError。提示：用 pytest.raises：

```
import pytest
from pydantic import ValidationError

def test_blank_model_is_rejected():
    with pytest.raises(ValidationError):
        CarInput(make="Mazda", model=" ", year=2020, color="Red")
```

第 2 层：集成测试（应用代码 + 真实 SQLite）

```
from app import database
from app.schemas import CarInput

def test_saved_car_can_be_read_from_sqlite(database_path):
    database.initialize(database_path)                    # 用临时数据库
    saved = database.create_car(
        database_path,
        CarInput(make="Toyota", model="Yaris", year=2020, color="Blue"),
```

```
    )
    loaded = database.get_car(database_path, saved.id)
    assert loaded is not None
    assert loaded.id == saved.id
    assert loaded.make == "Toyota"
    assert loaded.year == 2020
```

注意 database_path 参数：它就是 conftest.py 提供的临时数据库路径 fixture。这个测试还隐含验证了一件事：create_car() 和 get_car() 用的是两个不同的 SQLite 连接，能读回来证明写入真的提交并持久化了。

你的任务（test_deleting_one_car_keeps_the_other）：创建两辆车 → 只删第一辆 → 断言 get_car 找不到它、list_cars 恰好只剩第二辆。

第 3 层：API 测试（HTTP 契约）

API 测试属于集成测试的一种，用 FastAPI 的 TestClient 直接对应用发请求，不启动 Uvicorn、不开浏览器：

```
def test_post_car_returns_201_and_json(client):
    payload = {"make": "Ford", "model": "Fiesta", "year": 2018, "color": "Black"}
    response = client.post("/api/cars", json=payload)
    assert response.status_code == 201
    car = response.json()
    assert car["id"] > 0
    assert car["make"] == "Ford"
    assert response.headers["location"] == f"/api/cars/{car['id']}"
```

client 也是 fixture（conftest.py 定义）：它把 CARS_DB_PATH 指向临时数据库并创建 TestClient。你的任务（test_invalid_year_is_not_saved）：POST 一个 year=1800 的请求，断言返回 422，再 GET /api/cars 断言是空列表——校验失败时数据绝不能落库。

第 4 层：E2E 测试（真实浏览器）

E2E 由 Playwright 驱动真实 Chromium 浏览器走完整用户流程。tests/e2e/conftest.py 提供了两个 fixture：live_server（自动在空闲端口启动临时服务器）和 page（启动浏览器并打开新页面）。示例：

```
def test_user_creates_car_in_browser(page, live_server):
    page.goto(f"{live_server}/cars/new")
    page.get_by_label("Make", exact=True).fill("Toyota")
    page.get_by_label("Model", exact=True).fill("Yaris")
    page.get_by_label("Year", exact=True).fill("2020")
    page.get_by_label("Color", exact=False).fill("Blue")
    page.get_by_role("button", name="Create car").click()
    expect(page).to_have_url(re.compile(r"/cars/d+$"))   # 跳转到了详情页
    expect(page.get_by_role("heading", name="Toyota Yaris")).to_be_visible()
```

Playwright 的 API 很口语化：get_by_label 按表单标签找输入框，get_by_role 按按钮角色找按钮，fill 填内容，click 点击，expect 断言页面状态。装一次浏览器：python -m playwright install chromium（Linux 上还要加 --with-deps）。你的任务：写一个“创建 Honda Civic → 编辑为 Accord → 保存 → 刷新后更改仍在”的测试。

> **易错点**

> 四个测试组分别运行：python -m pytest tests/unit -q、tests/integration、tests/api、tests/e2e。E2E 失败先想 Chromium 装了没有。
> 预期看到 1 passed / 2 passed 却没通过时，先读最后一个断言附近的输出，不要被长长的 traceback 吓住。
> 故意改坏的断言在练习结束后一定要改回来——提交到仓库的必须是可通过的测试。

4.5 把测试流程搬到 GitHub（练习 2-1）

练习材料：练习指南/中文版/2-1. GITHUB_TESTING_WORKFLOW.pdf。流程：在 GitHub 建 cars-crud 空仓库 → 本地 git init 并推送应用 → 建 writing-tests 分支 → 在分支上完成上面全部测试练习 → 推分支 → 开 PR 合回 main。这一步把第三章的 Git 技能和本章的测试技能串了起来：

```
git switch -c writing-tests
# 完成四个层级的测试...
python -m pytest tests/unit tests/integration tests/api -q
python -m playwright install chromium
python -m pytest tests/e2e -q
git add .
git commit -m "Add unit, integration, API, and E2E tests"
git push -u origin writing-tests
# GitHub 网页：Compare & pull request → base: main, compare: writing-tests
# 合并要求：所有测试通过，提交的是恢复后的可通过版本
```

4.6 常见错误速查

现象	原因与对策
ModuleNotFoundError: No module named 'app'	终端目录不对，要在 cars-crud/ 下运行；或虚拟环境没激活
1 failed, AssertionError	对照 expected/actual 两行：actual 是实际值，expected 是断言值，差异一目了然
E2E 测试 ERROR，提示找不到浏览器	先运行 python -m playwright install chromium
Linux 上浏览器启动失败	用 python -m playwright install --with-deps chromium 装系统依赖
测试修改了真实 cars.db	正常不会发生——fixture 保证了临时数据库。检查是不是自己写了绕过 fixture 的代码
422 与 400 分不清	FastAPI/Pydantic 校验失败返回 422；数据格式根本解析不了才是 400

第五章 GitHub Actions 持续集成

5.1 CI 是什么、GitHub Actions 怎么工作

持续集成（CI, Continuous Integration）：每次有人推送代码或提交 PR，都自动在干净的机器上重新安装依赖、运行全部测试，第一时间发现“在我电脑上好好的”式问题。GitHub Actions 是 GitHub 内置的 CI 服务：

- 工作流（Workflow）是放在 .github/workflows/ 目录下的 YAML 文件；配置的事件发生时，GitHub 自动运行它。
- 事件（Event）如 push、pull_request；还可以 workflow_dispatch 允许在网页上手动触发。
- 作业（Job）在一台全新的虚拟机（运行器 Runner，如 ubuntu-latest）上运行，机器用完即弃，每次都是干净环境。
- 步骤（Step）是作业内的任务，按顺序执行；uses: 引用别人写好的可复用动作（action），run: 直接执行 shell 命令。

5.2 最小工作流逐键讲解（练习 3-1）

练习材料：练习指南/中文版/3-1. GITHUB_ACTIONS_GUIDE.pdf。第一个工作流 hello-python.yml 不长，逐行看懂它，后面就都通了：

```yaml
name: Hello Python                      # 显示在 Actions 标签页的名字，可自定义
on:
  push:
    branches: [main]                    # 推送到 main 时触发
  workflow_dispatch:                    # 允许网页手动运行
permissions:
  contents: read                        # 最小权限：只读仓库内容
jobs:
  hello:                                # 作业名，可自定义
    runs-on: ubuntu-latest              # 在什么系统上跑
    steps:
    - name: Download the repository
      uses: actions/checkout@v7         # 拉取仓库代码
    - name: Set up Python
      uses: actions/setup-python@v7
      with:
        python-version: "3.12"
    - name: Show Python version
      run: python --version             # 直接跑 shell 命令
    - name: Print a message
      run: python -c "print('Hello from GitHub Actions!')"
```

键	含义	能改吗
name	Actions 页面显示的名称	可以
on	触发事件；push/pull_request/workflow_dispatch	可以，注意 pull_request 的分支指目标分支
permissions	工作流令牌的权限，保持最小化	可以，但别放大
jobs / hello	作业分组与作业标识符	可以
runs-on	运行器系统，还有 windows-latest、macos-latest	可以
steps	按顺序执行的步骤	可以

键	含义	能改吗
uses	引用可复用动作（@v7 是主版本号）	可以换其他动作
with	给动作传参数	取决于动作
run	在运行器 shell 里执行命令；多行用 run: |	可以

易错点

YAML 缩进只能用空格。缩进错一格，整个工作流就解析失败——报“无效工作流”先检查空格。

工作流文件必须在仓库的默认分支（main）上才生效；改完要 commit + push。

workflow_dispatch（手动运行按钮）也要求文件在默认分支上。

做完 3-1 的验证步骤：push 后在 Actions 标签页打开运行，展开每个步骤看日志；把最后一个命令改成 python -c "raise SystemExit(1)" 再 push，观察红色失败，然后改回来。这个实验让你理解：run 的命令返回非零退出码，步骤就失败。

5.3 实战：PR 上自动跑 Cars 测试（练习 3-2）

练习材料：练习指南/中文版/3-2. GITHUB_ACTIONS_EXERCISE.pdf。要求：对以 main 为目标的 PR 自动运行四个测试组。参考实现（先自己填空，再看答案）：

```
name: Cars CRUD Tests
on:
  pull_request:
    branches: [main]        # main 是 PR 的目标分支，不是源分支
permissions:
  contents: read
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-python@v7
      with:
        python-version: "3.12"
    - name: Install dependencies
      run: python -m pip install -r requirements-dev.txt
    - name: Install Chromium
      run: python -m playwright install --with-deps chromium
    - name: Unit tests
      run: python -m pytest tests/unit -q
    - name: Integration tests
      run: python -m pytest tests/integration -q
    - name: API tests
      run: python -m pytest tests/api -q
    - name: E2E tests
      run: python -m pytest tests/e2e -q
```

要点：requirements-dev.txt 同时包含运行依赖和测试依赖，装它一个就够；E2E 在 Ubuntu 上需要 --with-deps 装浏览器系统库；一个步骤失败，后续步骤默认被跳过，作业标红。

验证方法（练习第 6 步很值得做）：故意把某个断言改坏并 push，看到 PR 的 Checks 里出现红色失败；改回来再 push，看到重新变绿。这就是 CI 的价值——烂代码在合并前就被拦下。

5.4 事件与分支过滤的细节

- push 的 branches 指“接收推送的分支”；pull_request 的 branches 指“PR 的目标（base）分支”。从 writing-tests 向 main 提的 PR 匹配 branches: [main]。
- PR 被打开、重新打开、追加提交时，pull_request 触发的工作流都会重新运行。
- 没有工作流运行时，依次检查：文件路径是不是 .github/workflows/？事件和分支匹配吗？仓库的 Actions 功能是否启用？

5.5 常见错误速查

现象	原因与对策
PR 上没有出现任何运行	文件路径不对、YAML 无效、Actions 被禁用，或 PR 目标不是 main
No module named pytest	测试前没装依赖：先 pip install -r requirements-dev.txt
Chromium executable missing	E2E 前没装浏览器：要用 install --with-deps chromium
后面的测试组没运行	前面的步骤已失败，作业在失败处停止。先看第一个红色步骤
手动运行按钮找不到	workflow_dispatch 没配置，或文件还不在默认分支上
本地能过、线上失败	线上是干净环境，缺什么依赖都会暴露——把缺失的包装进 requirements 再提交

第六章 Docker 容器化

6.1 为什么需要 Docker

传统部署的痛点：每换一台机器，都要手动装操作系统、语言环境、各种库和框架，步骤繁琐且极易出现“环境不一致”——开发机上行、服务器上就不行。Docker 的答案是“一次构建，到处运行”：把应用和它的全部依赖打包成一个标准单元（镜像 Image），这个单元在任意装了 Docker 的机器上都以完全一致的方式运行。设计思想来自海运集装箱：不管里面装什么，箱子是标准尺寸，任何港口、任何船都能装卸。

镜像与容器

术语	含义
镜像（Image）	打包好的“安装包”：应用代码 + Python + 依赖 + 最小运行系统
容器（Container）	镜像运行起来的实例；同一个镜像可以启动任意多个容器，互不影响
Docker Hub	官方公共镜像仓库，大量现成的镜像可直接拉取使用
Dockerfile	构建镜像的“配方文件”，一条条指令描述镜像怎么做出来

术语	含义
仓库（Registry）/ 标签（Tag）	存放镜像的在线服务；镜像的版本标签，如 cars-crud:v1
卷（Volume）	独立于容器的持久存储，容器删了数据还在

Docker 与虚拟机的区别

维度	虚拟机	Docker 容器
底层	通过 Hypervisor 分配硬件资源，独占一份宿主机配额	运行在 Docker Engine 上，不独占资源
操作系统	自带完整独立的操作系统	只含运行所需的最小组件
开销	重，一台机器开不了几个	轻，一台机器可同时跑大量容器

6.2 Dockerfile 逐行讲解（cars-crud）

练习材料：练习指南/中文版/4-1. DOCKER_GUIDE.pdf。项目根目录的 Dockerfile 全文：

```
ARG BASE_IMAGE=python:3.12-slim   # 构建参数：基础镜像可替换（国内加速用）
FROM ${BASE_IMAGE}                  # 第一层：从官方 Python 3.12 slim 镜像开始
WORKDIR /app                        # 设置镜像内的工作目录
COPY requirements.txt ./            # 先复制依赖清单...
RUN python -m pip install --no-cache-dir -r requirements.txt   # ...并安装（构建期执行）
COPY app/ ./app/                    # 再复制应用代码
RUN mkdir -p /data && chown 10001:10001 /data   # 建数据目录并授权给非 root 用户
USER 10001:10001                    # 以无特权用户运行，安全惯例
ENV CARS_DB_PATH=/data/cars.db      # SQLite 路径指向容器内 /data
EXPOSE 8000                         # 声明容器内端口（只是声明，不自动发布）
CMD ["python", "-m", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

四个必须理解的点：

1. RUN 与 CMD 的时机不同：所有 RUN 在构建镜像时执行一次（装包、建目录）；CMD 在容器每次启动时执行（启动服务器）。混淆它们是高频错误。
2. CMD 参数用 JSON 列表写法，命令不会被 shell 展开。
3. 先 COPY requirements.txt 再 COPY 代码：利用镜像分层缓存，只改代码时不必重装依赖，构建更快。只装 requirements.txt（运行依赖）而不是 -dev（测试工具），镜像更小。
4. --host 0.0.0.0：容器内的 127.0.0.1 只有容器自己访问得到；绑定 0.0.0.0 外部映射进来的请求才能到达。本地开发用 127.0.0.1 没问题，进容器必须改。

.dockerignore 与 .gitignore 同理：把 .git/、.venv/、tests/、*.db 等排除在构建之外，既减小体积也避免把本地数据库和敏感文件打进镜像。

6.3 构建、运行、端口映射与卷（练习 4-1 主体）

前提：docker --version 和 docker info 都正常（Docker Desktop 已启动）。检查没问题后：

```
# 构建镜像（最后的 . 表示用当前目录作为构建上下文）
docker build -t cars-crud:local .

# 启动容器：-d 后台，--name 起名，-p 做端口映射，--mount 挂数据卷
docker run -d --name cars-app -p 127.0.0.1:8000:8000 \n --mount
type=volume,source=cars-data,target=/data cars-crud:local

# 打开 http://localhost:8000/ 和 http://localhost:8000/docs 验证
```

端口映射 -p 127.0.0.1:8000:8000 的意思是：宿主机（你的电脑）的 8000 端口 → 容器的 8000 端口。容器内部是一个隔离网络，不从外面映射就访问不到。同一台机器跑多个容器时，容器内部都可以用 8000，只要宿主机的映射端口（8001:8000、8002:8000……）互不冲突。

数据卷验证（理解卷的意义）：

```
docker stop cars-app && docker rm cars-app     # 删除容器
docker run -d --name cars-app -p 127.0.0.1:8000:8000 \n --mount
type=volume,source=cars-data,target=/data cars-crud:local
# 刷新应用：之前创建的车还在！SQLite 数据保存在卷里，与容器生命周期无关
```

Docker CLI 速查表

命令	用途
docker build -t 名称 .	按当前目录的 Dockerfile 构建镜像
docker image ls	列出本地镜像
docker ps / docker ps -a	列出运行中 / 全部容器
docker logs -f cars-app	跟踪容器日志，Ctrl+C 停止跟踪
docker stop / start / rm cars-app	停止 / 重启 / 删除容器（rm 不删卷）
docker exec -it cars-app sh	进容器内部开一个 shell
docker volume ls	列出卷
docker tag 源 目标	给镜像打新标签
docker push / pull 镜像	上传 / 下载镜像

6.4 国内加速：课程 Azure 代理

国内直接访问 Docker Hub 拉取镜像很慢。课程提供了 Azure 代理：拉取时在镜像名前加前缀即可——

```
# 普通拉取
docker pull python:3.12
# 通过课程代理拉取同样的镜像
docker pull efreiwut.azurecr.io/dockerhub/library/python:3.12
# 构建时临时换基础镜像（Dockerfile 的 ARG 就是为此设计的）
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/dockerhub/library/python:3.12-slim -t cars-
crud:local .
```

易错点

> 代理只是“拉取基础镜像”的加速通道；发布自己镜像用的仍然是你的 Docker Hub 仓库，两者别混淆。
>
> :3.12 和 :3.12-slim 是两个不同的镜像标签；slim 版体积小得多（Python 完整版约 1.6GB，slim 约 250MB）。
>
> GitHub Actions 的服务器在国外，不需要代理，直接用默认的 python:3.12-slim。

6.5 发布到 Docker Hub（练习 4-1 后半）

1. 在 hub.docker.com 注册账号，记住 Docker ID；My Hub → Repositories → Create repository 建一个名为 cars-crud 的仓库（Public 即可）。
2. Account settings → Personal access tokens 生成一个具有 Repo Read & Write 权限的令牌，复制保存（只显示一次）。这个令牌稍后在 GitHub Actions 练习里还会用到。
3. 终端登录：docker login --username YOUR_DOCKER_ID，密码处粘贴令牌（不是 GitHub 密码，也不是 Docker 登录密码）。
4. 打标签并推送：

```
docker tag cars-crud:local YOUR_DOCKER_ID/cars-crud:v1
docker push YOUR_DOCKER_ID/cars-crud:v1
# 命名格式：NAMESPACE/REPOSITORY:TAG
# 换台电脑（或删掉本地镜像后）验证：
docker pull YOUR_DOCKER_ID/cars-crud:v1
```

6.6 用 GitHub Actions 自动发布（练习 4-2）

练习材料：练习指南/中文版/4-2. DOCKER_PUBLISHING_EXERCISE.pdf。目标：扩展第五章的 tests.yml，让它同时满足——PR 只跑测试不发镜像；合并到 main 后先跑测试，全绿才构建并推送镜像到你的 Docker Hub。

第一步：在 GitHub 配置 secrets

Settings → Secrets and variables → Actions → New repository secret，建两个：DOCKERHUB_USERNAME（你的 Docker ID）和 DOCKERHUB_TOKEN（6.5 节生成的令牌）。令牌绝不写进 YAML、.env 或截图——secrets 就是为此存在的。

第二步：改造工作流

```
name: Cars CRUD Tests
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]          # 合并 PR 会在 main 上产生一次 push
permissions:
  contents: read
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v7
```

```
    - uses: actions/setup-python@v7
      with:
        python-version: "3.12"
    - run: python -m pip install -r requirements-dev.txt
    - run: python -m playwright install --with-deps chromium
    - run: python -m pytest tests/unit -q
    - run: python -m pytest tests/integration -q
    - run: python -m pytest tests/api -q
    - run: python -m pytest tests/e2e -q
  publish:
    needs: tests                # 等 tests 成功才运行
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v7
    - uses: docker/setup-buildx-action@v4
    - uses: docker/login-action@v4
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}
    - uses: docker/build-push-action@v7
      with:
        context: .
        push: true
        tags: YOUR_DOCKER_ID/cars-crud:latest
```

两个新知识点：

- needs: tests 让 publish 作业等 tests 成功；任何一个测试失败，发布自动被跳过——这就是“测试是发布的门禁”。
- if 条件 github.event_name == 'push' && github.ref == 'refs/heads/main' 确保发布只在 main 分支的 push 上发生。PR 上绝对不用发布凭据：未合并的 PR 代码可能来自任何人，把 Docker Hub 令牌暴露给它等于把家门钥匙交给陌生人；而且 fork 的 PR 本来也读不到仓库 secrets。

验证闭环

1. PR 上：Checks 显示测试通过、publish 被跳过（Skips 是正常的）。
2. 合并 PR → main 上触发新运行 → tests 先绿 → publish 登录 Docker Hub、构建、推送。
3. 打开 Docker Hub 仓库确认 latest 标签出现；本地 docker pull + docker run 跑起来，行为和本地构建完全一致。
4. 可选加分项：除 latest 外再推一个带 Git 提交 SHA 的标签（如 :sha-abc1234），方便把镜像和具体提交对应起来。

6.7 Docker 常见错误速查

现象	原因与对策
无法连接到 Docker 守护进程	Docker Desktop 没启动；先启动再执行命令
找不到 Python 模块	requirements.txt 或 app/ 不在构建上下文里；从项目根目录构建，检查 .dockerignore 是否误排除
浏览器连不上应用	docker ps 看容器是否在跑；docker logs cars-app 看报错；-

现象	原因与对策
	p 映射对不对
8000 端口已被占用	本地 Uvicorn 还开着；停掉它或改用 127.0.0.1:8001:8000
Permission denied 写 SQLite	用挂在 /data 的命名卷（参考 Dockerfile 的 USER 10001 设计），别用需要调所有权的自定义 bind 挂载
推 Docker Hub 被拒绝	检查仓库名、令牌权限（Repo Read & Write）、docker login 是否过期
Azure 代理拉取失败	核对 efreiwut.azurecr.io/dockerhub/library/ 路径和镜像标签是否写对

第七章 全链路串讲与复习

7.1 从写代码到上线：一张总图

把第三到第六章串起来，一个功能从开发到上线的完整旅程是：

1. 开发：从 main 切出功能分支，写代码 + 写测试（第三章 + 第四章）。
2. 提交：add → commit → push 到远程分支（第三章）。
3. 开 PR：GitHub Actions 自动在干净环境装依赖、跑四层测试（第四章 + 第五章）。
4. 评审与合并：测试全绿才可合并进 main（第三章 + 第五章）。
5. 发布：合并触发 main 上的 push，Actions 自动构建 Docker 镜像并推送到 Docker Hub（第六章）。
6. 部署与运行：任何机器 docker pull 镜像、docker run 即得到与开发一致的应用（第六章）。
7. 监控与迭代：线上反馈进入下一轮规划——回到第 1 步（第一章）。

这就是课程一开始讲的 DevOps 生命周期在课程项目上的完整落地。期末项目要求三人组队复现这条流水线，可以把这个列表当作项目分工的检查单。

7.2 全课程命令总表

Git

场景	命令
开始新项目	git init -b main / git clone <url> / git remote add origin <url>
日常提交	git status → git add <file> → git commit -m "信息" → git push
观察	git diff [--staged] / git log --oneline [--graph --all]
分支	git switch -c 新分支 / git switch main / git branch / git merge 分支
同步	git fetch origin / git pull [--rebase] / git push -u origin 分支

场景	命令
撤销	git restore --staged <file> / git restore <file> / git revert HEAD / git reset --soft HEAD~1
暂存	git stash push -m "说明" / git stash list / git stash pop

测试

场景	命令
装依赖	pip install -r requirements-dev.txt
启动应用	python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
跑测试	python -m pytest -q（全部）/ python -m pytest tests/unit -q（单层）/ -v（详情）
装浏览器	python -m playwright install chromium（Linux 加 --with-deps）

Docker

场景	命令
构建	docker build -t cars-crud:local .（代理：--build-arg BASE_IMAGE=efreiwut.azurecr.io/dockerhub/library/python:3.12-slim）
运行	docker run -d --name cars-app -p 127.0.0.1:8000:8000 --mount type=volume,source=cars-data,target=/data cars-crud:local
管理	docker ps -a / docker logs -f cars-app / docker stop|start|rm cars-app / docker volume ls
发布	docker login → docker tag cars-crud:local ID/cars-crud:v1 → docker push ID/cars-crud:v1

7.3 自测题（附答案）

概念题

1. DevOps 的四大衡量指标是什么？分别往什么方向优化？
2. Git 中工作区、暂存区、提交三者是什么关系？git add 和 git commit 各做了什么？
3. merge 和 rebase 的本质区别是什么？为什么已发布的提交不能 rebase？
4. 单元测试、集成测试、E2E 测试各测什么？为什么数量是金字塔分布？
5. fixture 是什么？课程项目里 database_path fixture 解决了什么问题？
6. GitHub Actions 里 jobs、steps、uses、run 各是什么？needs 和 if 各起什么作用？
7. 镜像和容器的区别？RUN 和 CMD 的区别？为什么要做端口映射和数据卷？
8. 为什么发布 Docker 镜像的作业不能在 PR 上运行？

参考答案

1）部署频率（越高越好）、变更交付周期（越短越好）、变更失败率（越低越好）、故障检测与修复平均耗时（越短越好）。2）工作区是磁盘上的文件，add 把改动放进暂存区，commit 把暂存区保存为带 ID 的快照。3）merge 汇合历史、可能产生合并提交；rebase 把提交搬到新基线上、改写提交 ID——已发布的提交被别人基于它工作，改写会让所有人的历史错乱。4）单元测单个函数（不碰数据库网络），集成测组件协作，E2E 模拟真实用户全链路；越靠近底层越便宜越稳定，所以数量上单元 > 集成 > E2E。5）fixture 是测试前自动准备的道具；database_path 让每个测试拥有独立临时数据库，不碰真实数据、互不干扰。6）job 是一台机器上的一组步骤，steps 按顺序执行；uses 引用可复用动作，run 执行 shell 命令；needs 控制作业间依赖顺序，if 控制作业运行条件。7）镜像是打包好的安装包，容器是镜像的运行实例；RUN 构建期执行、CMD 启动期执行；容器网络隔离所以要 -p 端口映射，容器删除数据即失所以要卷持久化。8）PR 代码未经验证可能来自任何人，把 Docker Hub 令牌交给它是安全风险；且 fork 的 PR 拿不到仓库 secrets。

7.4 考前与项目前检查清单

- 能不看文档完成：建仓库、提交、分支、合并、解决一次冲突、推送、开 PR。
- cars-crud 四个测试组本地全绿，并能说出每层在测什么。
- PR 上 Actions 自动跑测试，故意改坏一个断言能看到红叉，修复后变绿。
- 本地 docker build + docker run 跑通应用，创建的车在删容器重建后还在。
- 合并到 main 后 Actions 自动发布 latest 镜像到 Docker Hub，拉下来能跑。
- 能向组员解释 7.1 那张总图的每一步在做什么、为什么。

全部打勾，说明你已经掌握了课程前四天的全部内容，可以从容迎接 MLOps 部分和期末项目。祝学习顺利！